Informatik I

Dr. Henrik Brosenne
Georg-August-Universität Göttingen
Institut für Informatik

Wintersemester 2025/26

Inhaltsverzeichnis

1 Einführung **1**
1.1 Informatik und Programmierung 1
1.2 Python 3
1.3 Problemlösung in der Informatik 3
1.3.1 Beispiel: Einfache arithmetische Reihe 5
1.4 Problem, Algorithmus, Programm 9

A Anhang **19**
A.1 Darstellung von Fachbegriffen 19
A.2 Python Operators 20
A.3 Python Keywords 24

Schlagwortverzeichnis **25**

Abbildungsverzeichnis

1 Jupyter Notebook Documentation 4
2 First Steps 5

1 Einführung

1.1 Informatik und Programmierung

Das Wort **Informatik** ist vergleichsweise jung. Es entstand in den 1960er-Jahren in Anlehnung an das französische *informatique*, zusammengesetzt aus *Information* und *Automatik*. Gemeint ist die systematische Verarbeitung von Informationen – nicht unbedingt mit der Hilfe von Maschinen. Dazu gehört die Abstraktion durch Modelle, sowie die Entwicklung von Methoden (z.B. Algorithmen und Datenstrukturen) und deren Bewertung (z.B. Korrektheit und Aufwand).

Die Probleme, mit denen sich die Informatik beschäftigt, sind oft viel älter. Schon im Altertum wurde über effiziente Zahlensysteme gestritten, das indische Stellenwertsystem machte später vieles einfacher. In der Renaissance entwarf Wilhelm Schickard eine Rechenmaschine, und Leibniz träumte davon, das logische Schlussfolgern zu mechanisieren.

Informatik ist relativ jung als Wissenschaft, aber oft alt in ihren Fragen.

Die Vorlesung wird eine Einführung in die Grundlagen der (modernen) Informatik bieten, dabei aber häufig auf alte gelöste und ungelöste Probleme zurückgreifen.

Programme gab es lange vor Computern. Programmieren im Kern nichts anderes ist als das präzise Formulieren und Abarbeiten von **Algorithmen** – und die Idee ist uralt.

Nachfolgende ein paar willkürlich ausgewählte Stationen.

Antike Mathematik

- **Euklid** (ca. 300 v. Chr.) beschreibt den *Algorithmus zur Berechnung des größten gemeinsamen Teilers (ggT)* – ein Verfahren, das wir im Prinzip heute noch genauso in Code schreiben können.
- Die Babylonier und Ägypter hatten Rechenverfahren für Multiplikation, Wurzeln etc., die streng schrittweise notiert sind.

Mittelalterliche Rechenkunst

- **Al-Chwarizmi** (9. Jh.), nach ihm ist das Wort **Algorithmus** benannt, systematisiert schriftliche Rechenverfahren.

- Rechenmeister in Europa führten später Rechenbücher, die wie Kochrezepte für Zahlen aussahen.

Mechanische Automatisierung

- Der antike **Antikythera-Mechanismus** (ca. 100 v. Chr.) rechnete Planetenpositionen mechanisch aus. Der Mechanismus war nicht programmierbar, aber ein frühes Beispiel für Automatisierung.
- Im 18./19. Jahrhundert dann **Jacquards Webstuhl**: Lochkarten steuern das Muster – praktisch ein Programm für Textilien.

Formalisierung

- **Ada Lovelace** schrieb 1843 das erste **Programm** für eine programmierbare Maschine. Die **Analytical Engine** von Charles Babbage wurde aber nie gebaut.
- **George Boole** (19. Jh.) entwickelt die Boolesche Algebra – die Grundlage heutiger Schaltkreise und Programmsteuerung.

Computer

- Mit Relais, Röhren und Lochkarten entstanden Computer wie der **Zuse Z3** (1941), oft als erster funktionsfähiger, programmierbarer Rechner bezeichnet. Bald folgten in den USA ENIAC und EDVAC. Damit wurde aus Theorie Praxis – Computer, die wirklich Programme ausführen könnten.

Computer Science is no more about computers than astronomy is about telescopes. *Edsger W. Dijkstra*

Algorithmus

Algorithmus

= endliche, schrittweise, präzise (Handlung-/Rechen-)Vorschrift
- **endlich**: endlicher Text, Diagramm, ...
- **schrittweise**: einfache Einzelaktionen
- **präzise**: Schritte+Reihenfolge unmissverständlich, vollständig

Beispiele und Gegenbeispiele

- schriftliche Multiplikation
- bebilderte Möbelbauanleitung (unklare Reihenfolge, z.T. komplexe Einzelaktionen)
- Kochrezept (komplexe Einzelaktionen, unpräzise)

1.2 Python

In dieser Vorlesung wird die Programmiersprache **Python** benutzt.

Eine **Programmiersprache** ermöglicht es, Algorithmen präzise und eindeutig zu formulieren. Für produktiv genutzte Programmiersprachen, wie z.B. Python, gibt es zudem *tools* (Werkzeuge), um Programme zu überprüfen, Fehler zu finden und ihre Funktionsweise durch Tests nachvollziehbar zu machen.

Ein mächtiges *tool* sind **Jupyter Notebooks**. Eine interaktive Umgebung, in der sich Code, Texte, Formeln und Grafiken in kombinieren lassen.

Die *Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen (GWDG)* stellt mit der **GWDG Jupyter Cloud** ⟨`https://jupyter-cloud.gwdg.de/`⟩ allen Studierenden einen einfachen Zugang zu Jupyter Notebooks zur Verfügung.

Die Jupyter Notebooks der Vorlesung werden in der Jupyter Cloud geteilt.

1.3 Problemlösung in der Informatik

Probleme und ihre Lösung

Problem

Eine **Problem** ist eine **allgemeine** Aufgabenstellungen folgender Form.
gegeben *input* (Eingabe) aus einer Menge möglicher *inputs*
gesucht *process* (Verarbeitungvorschrift), d.h. ein **Algorithmus** der jeden gegebenen *input* zu dem jeweils erwarteten *output* verarbeitet
erwartet *output* (Ausgabe) für jeden möglichen *input*

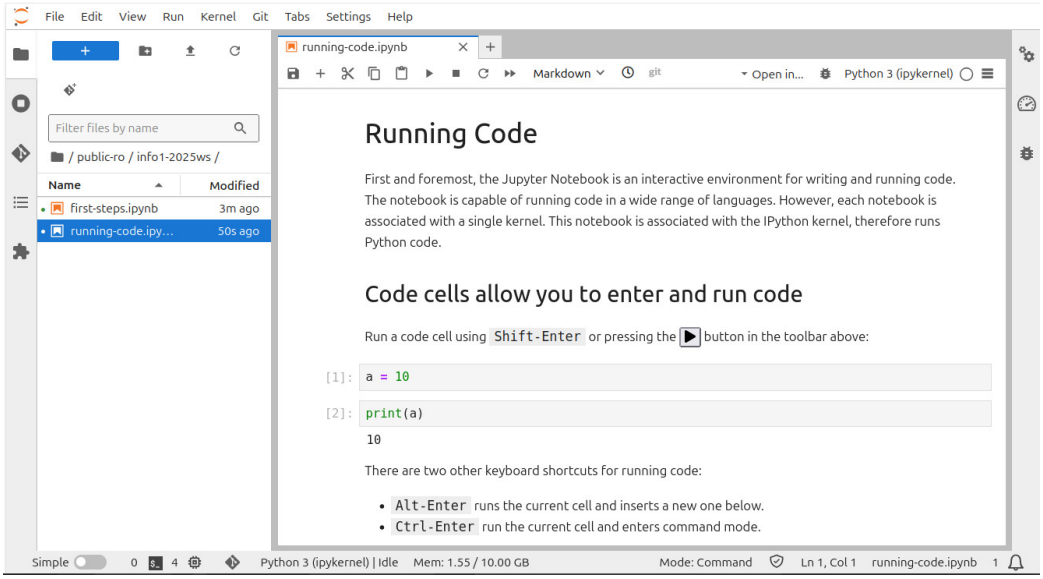


Abbildung 1: Jupyter Notebook Documentation
(Quelle: ⟨`https://jupyter-notebook.readthedocs.io/en/latest/`⟩)

Algorithmus

In der Informatik sucht man nach *guten* Algorithmen, bei unterschiedlichsten Anforderungen.

Beispiele

- effizient, bezogen auf Rechenzeit und/oder Speicherbedarf
- kostengünstig in Entwicklung und Betrieb
- sicher, Stichwort Datenschutz
- ...

IPO model

Der so entstehende Ablauf ist ein Grundprinzip der Datenverarbeitung, das *Input–Process–Output model* (Eingabe-Verarbeitung-Ausgabe Prinzip), kurz *IPO model* (EVA-Prinzip).

IPO model



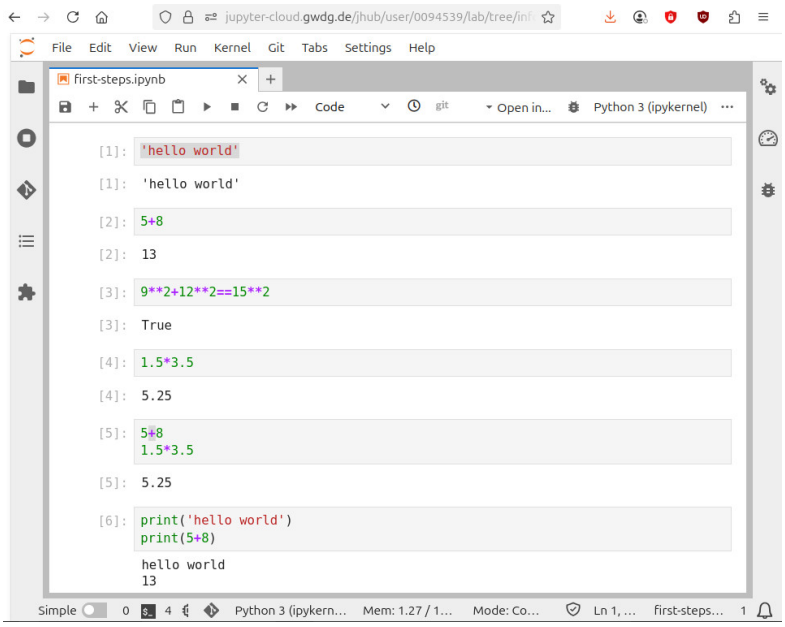


Abbildung 2: First Steps

1.3.1 Beispiel: Einfache arithmetische Reihe

Einfache arithmetische Reihe

input Zahl n aus den natürlichen Zahlen $\mathbb{N}$
process Algorithmus der $\sum_{i=1}^{n} i$ für alle $n \in \mathbb{N}$ berechnet
output Summe der ersten n aufeinanderfolgenden natürlichen Zahlen

Ein möglicher Algorithmus ist die Berechnung mit Hilfe einer Tabelle.

Algorithmus 1

Lege für die Zahlen 1 bis n jeweils eine Spalte an, initialisiere Spalte 1 mit 1.

Gehe die Spalten beginnend bei 2 bis n durch, notiert in Spalte i den Wert aus Spalte $i-1$ plus i.

Gibt den Eintrag der Spalte n aus.

1	2	3	4	5	...	45	46	...	98	99	100	...
1	3	6	10	15	...	1035	1081	...	4851	4950	5050	...

Diesen Algorithmus kann ein Mensch, sogar ein Grundschulkind, leicht um-

setzen, solange n nicht zu groß wird. Spaß macht das allerdings nicht und genau diese *Fleißarbeit* kann man an einen Computer delegieren.

Vor der Umsetzung in eine Programmiersprache lohnt es sich den Algorithmus etwas zu analysieren.

- der *output* steht nur in der letzten Spalte
- für die Berechnung des Eintrags einer Spalte benötigt man nur den Eintrag in der unmittelbar davor stehenden Spalte

Mit diesen Beobachtungen kann man auf die Tabelle verzichten und es reicht ein Speicherplatz für den Index der aktuellen Spalte und ein weiterer Speicherplatz für den Eintrag in der vorherigen Spalte, der immer wiederverwendet wird.

Algorithmus 2

Der Index wird mit `i = 1` und die Summe mit `series = 0` initialisiert.

Wiederhole folgendes, solange für den Index `i <= n` gilt.

- aktualisiere die Summe `series = series + i`
- aktualisiere den Index `i = i + 1`

Gibt die Summe `series` aus.

Hinweis

Der naheliegendere Bezeichner `sum` für die Summe ist der Name einer *Python Built in Function*, siehe ⟨`https://docs.python.org/3/library/functions.html`⟩.

Zum Wiederholen von Anweisungen bietet Python *loops* (Schleifen) an.

Ein `while`-*loop* wiederholt die folgenden, gleich eingerückten Anweisungen, solange die *loop condition* (Fortführbedingung) erfüllt ist.

```
1  n = 100                # input
2  i = 1                  # init
3  series = 0
4  while i <= n:          # loop
5      series += i            # series = series + i
6      i += 1                 # i = i + 1
7  print(series)          # output
```

```
1  5050
```

Hinweis

Python kennt diverse Operatoren, z.B. *arithmetic operators* (arithmetische Operatoren), *assignment operators* (Zuweisungsoperatoren) und *comparison operators* (Vergleichsoperatoren).

Siehe A.2 und ⟨`https://www.w3schools.com/python/python_operators.asp`⟩.

Handelt es sich auch um einen **guten** Algorithmus?

- **Korrektheit**. Es ist leicht zu sehen, dass der Algorithmus korrekt ist, *process* wird wie beschriebenen umgesetzt.
- **Speicherbedarf**. Es werden drei Variablen (Speicherplätze) benutzt, unabhängig davon wie groß der Wert vom *input* n ist.
- **Laufzeit**. Zeitmessungen können wir (noch) nicht vornehmen. Deshalb benutzen wir eine andere Metrik, die Anzahl der durchgeführten Python-Operationen.

Durchgeführte Operationen

```
1  # input
2  n = 100                # 1x assignment
3
4  # process - init
5  i = 1                  # 2x assignment
6  series = 0
7
8  # process - loop
9                         # n interations
10 while i <= n:              # 1x comparison
11     series += i            # 2x assignment
12     i += 1
13
14 # output
15 print(series)
```

Die Anzahl der durchgeführten Operationen hängt vom *input* n ab und beträgt $3n + 3$.

Hinweis. In Python ist += ein *assignment operator*, deshalb zählen wir nur eine Operation.

Damit könnte man schon zufrieden sein. Aber hier bietet sich ein noch effizienterer Algorithmus an. Um diesen auszuarbeiten benötigen wir keinen Computer.

Wenn man die Zahlen 1 bis n zweimal untereinander notiert und zwar einmal aufsteigend und einmal absteigend, beträgt die Summe jeder Spalte $n + 1$.

Summiert man jetzt die Ergebnisse in den n Spalten, erhält man das Doppelte der Summe der ersten n natürlichen Zahlen, weil die Addition kommutativ und assoziativ ist.

	1	2	3	$\dots$	i	$\dots$	$n-2$	$n-1$	n
$+$	n	$n-1$	$n-2$	$\dots$	$n-i+1$	$\dots$	3	2	1
	$n+1$	$n+1$	$n+1$	$\dots$	$n+1$	$\dots$	$n+1$	$n+1$	$n+1$

Daraus folgt $n \cdot (n+1) = 2 \sum_{i=1}^{n} i$.

Hinweis

Vergleiche *triangular number* (Dreieckszahl) ⟨`https://en.wikipedia.org/wiki/Triangular_number`⟩.

Algorithmus 3

Gib $\frac{n \cdot (n+1)}{2}$ aus.

Hinweis

Dieser Algorithmus zur Berechnung der n-ten *triangular number* war vermutliche schon den *Pythagoräern* (6. Jh. v. Chr.) bekannt. Definitive nachgewiesen ist er für *Diophantos von Alexandria* und *Iamblichos von Chalkis* (3. Jh. v. Chr.).

Carl Friedrich Gauß (1777-1855) wird heute oft mit diesem Algorithmus (**Gaußsche Summenformel**) in Verbindung gebracht, aufgrund einer Anekdote, die er wohl selbst häufig erzählt hat, in der Gauß bereits als Kind die Formel verwendete.

Es bietet sich an diesen Algorithmus so zu implementieren, dass er leicht wiederverwendbar ist - ohne *copy and paste* (Kopieren und Einfügen).

Eine *function* (Funktion) definiert man in Python mit dem *keyword* (Schlüsselwort) `def`. Siehe A.3) für eine Liste der *Python keywords*.

Der *return value* (Rückgabewert) ist der Wert des Ausdrucks nach dem *keyword* `return`.

Eine *function* ruft man über den mit `def` vergebenen Namen auf.

```
1 def gauss_formula(n):
2     return n*(n+1)//2
3
4 gauss_formula(100)
```

```
1 5050
```

Handelt es sich um einen **besseren** Algorithmus?

- **Korrektheit**. Nach Konstruktion ist der Algorithmus korrekt.
- **Speicherbedarf**. Es wird nur eine Variablen benutzt, unabhängig davon wie groß der Wert vom *input* n ist.
- **Laufzeit**. Die Anzahl der durchgeführten Operationen beträgt 3, ebenfalls unabhängig von n.

Damit hat der Algorithmus 3 - mit unserer Metrik - die Laufzeit 3, das ist wesentlich besser als die Laufzeit $3n+3$ des Algorithmus 2. Insbesondere für große n.

1.4 Problem, Algorithmus, Programm

Was bedeutet „ein Algorithmus ist korrekt“?

Korrektheit

Ein Algorithmus

- **terminiert** für einen *input*, wenn er nach einer endlichen Anzahl von Verarbeitungsschritten einen *output* liefert.
- ist **partiell korrekt**, wenn er den erwarteten *output* liefert, falls er terminiert.
- ist (total) **korrekt**, wenn er partiell korrekt ist und für jeden zulässigen *input* terminiert.

Eine Korrektheitsüberprüfung erfordert eine genau **Spezifikation** des Problems.

- erlaubter *input*/erwarteter *output*, z.B. in sprachlicher Form, als mathematische Formel.
- zulässige *tools* (Werkzeuge), z.B. Ablaufsteuerung (*loops*), arithmetische Operationen (`+`, `-`).
- Verhalten bei Fehlern, z.B. unzulässigen Eingaben.
- ...

Erster Schritt zur Lösung eines Problems ist eine gute Spezifikation.

- unzweideutig
- vollständig
- (idealerweise) einfach und kurz

Beispiel: Unzureichende Spezifikation

gesucht Lösungen der Gleichung $a_2 \cdot x^2 + a_1 \cdot x + a_0 = 0$

Welche Größen sind gegegeben? Was sind die Wertebereiche? Wie ist mit Sonderfällen zu verfahren (z.B. keine reelle Lösung)?

Beispiel: Verbesserte Spezifikation

input drei Koeffizienten $a_0, a_1, a_2 \in \mathbb{R}$
output alle $x \in \mathbb{R}$ mit $a_2 \cdot x^2 + a_1 \cdot x + a_0 = 0$

Greatest Common Divisor

Der *greatest common divisor* (größte gemeinsame Teiler), abgekürzt *GCD*, zweier natürlicher Zahlen ist die größte natürliche Zahl, durch die sich beide Zahlen ohne Rest teilen lassen.

Beispiel. $\mathrm{GCD}(48, 27) = 3$

GCD-Problem

input natürliche Zahlen a, b
output eine natürliche Zahl d mit folgenden Eigenschaften
- d teilt a und b
- es gibt keinen Teiler d' von a und b mit $d' > d$

tools Arithmetik in den natürliche Zahlen

Eine Lösung des GCD-Problems

Idee in Prosa

> Wenn CD aber AB nicht misst, und man nimmt bei AB, CD abwechselnd immer das Kleinere vom Größeren weg, dann muss (schließlich) eine Strecke übrig bleiben, die die vorangehende misst.

⟨https://de.wikipedia.org/wiki/Euklidischer_Algorithmus⟩

Strukturierte Prosa (Pseudocode)

Euklidischer Algorithmus (klassisch)
input $a, b \in \mathbb{N}$
init $x \leftarrow a, y \leftarrow b$
iterate solange $x \neq y$
  Falls $(x > y)$ $x \leftarrow x - y$
  sonst $y \leftarrow y - x$
finalize liefere x
output x

$A \leftarrow B$ steht für: speichere aktuellen Wert von B in A

Eine Lösung des GCD-Problems

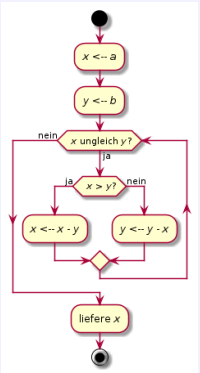
Der klassische Euklidische Algorithmus hat eine typische Struktur.

Iteratives Grundmuster

Init(ialize) - Iterate - Finalize

Ablauf des Algorithmus

Ablaufdiagramm



Prozess

Vorgang einer algorithmisch ablaufenden Informationsverarbeitung (nach Informatik-Duden)

Dynamisches Verhalten

situationsbedingte Reihenfolge der Einzelschritte

Ablaufverfolgung (Tracing)

Buchführung über die Variablenwerte
Beispiel: Tabelle mit Werten nach jedem Schritt

Algorithmus in Programmcode

Für die programmiersprachliche Notation eines Algorithmus in **Programmcode**, z.B. in Python, muss die Spezifikation auch passend formuliert sein.

Python kennt keine natürlichen Zahlen, aber *integer* (ganze Zahlen), deshalb ändern wir die Spezifikation entsprechend.

GCD-Problem

input *integer* $a, b > 0$
output *integer* $d > 0$ mit folgenden Eigenschaften
- d teilt a und b
- es gibt keinen Teiler d' von a und b mit $d' > d$

tools Ablaufsteuerung und Arithmetik in Python

Algorithmus in Python-Code

```
x = a                # assign input x to a
y = b                # assign input y to b
while (x != y):      # `!=` stands for `not equal`
    if (x > y):
        x = x - y    # calculate x - y, assign result to x
    else:
        y = y - x    # (analogous)
print(x)             # output
```

Trace Table

Trace Table

Eine *trace table* (Ablaufprotokoll) beschreibt zeilenweise den Ablauf eines Algorithmus.

Es werden die Reihenfolge, in der die Codezeile ausgeführt werden, und die Werte der (relevanten) Variablen **nach** der Ausführung der jeweiligen Codezeile, aufgelistet.

Beispiel

```
x = a
y = b
while (x != y):
    if (x > y):
        x = x - y
    else:
        y = y - x
print(x)
```

input $a = 30, b = 12$

Zeile(n)	x	y	
1,2	30	12	input: 30, 12
3,4	30	12	
5	18	12	
3,4	18	12	
5	6	12	
3,4	6	12	
7	6	6	
3	6	6	
8	6	6	output: 6

Loop Invariant

Die Korrektheit eines iterativen Algorithmus, bei dem *iterate* mit einem *loop* realisiert wird, lässt sich zeigen, indem man wiederkehrende Eigenschaften des *loop* betrachtet.

Loop Invariant

Eine *loop invariant* (Schleifeninvariante) ist eine Eigenschaft, die bei jedem Durchlauf des *loop* (an derselben Stelle) gilt.

Algorithmus mit *Loop Invariant*

init *input* $a, b \in \mathbb{Z}$, $a, b \geq 1$

Sei $\mathrm{GCD}(a, b) = d$, setze $x = a,\ y = b$ dann gilt $x = dx',\ y = dy'$.

iterate *loop*

- Fall $x = y$, d.h. $x = y = d$, weiter bei *finalize*.
- Fall $x > y$, dann folgt $x' > y'$ und $dx' - dy' = d \cdot \underbrace{(x' - y')}_{\geq 1}$.

  loop invariant

  - $d \cdot (x' - y') \geq 1$
  - $|d \cdot (x' - y') - y| < |x - y|$

– $\mathrm{GCD}(d \cdot (x' - y'), y) = \mathrm{GCD}(x, y)$

Setze $x = x - y = d \cdot (x' - y')$, weiter bei *iterate*.

- Fall $x < y$, analog.

finalize *output* x

Bemerkung

$$|z| = \begin{cases} -z & \text{wenn } z < 0 \\ z & \text{sonst} \end{cases} \qquad \text{(gesprochen: „Betrag von } z\text{“)}$$

Korrektheit mit *Loop Invariant*

Aus der *loop invariant* folgt die **Korrektheit**.

- **Terminiert.** Abstand zwischen x und y wird in jedem Schleifendurchlauf kleiner. Weil x und y positive ganze Zahlen sind bedeutet das, der Fall $x = y$ wird erreicht.
- **Partiell korrekt**. Das *update* (Aktualisieren) von x verändert die Problemstellung nicht, der GCD d des „alten“ und des „neuen“ x mit y ist identisch. Terminiert der Algorithmus liefert der triviale Fall $x = y = d$ den erwarteten *output*.

Laufzeit

Weder theoretisch noch praktisch können wir zu diesem Zeitpunkt die Laufzeit des GCD ermitteln.

Schränken wird die Eingabe ein, z.B. auf $a, b < 100$, ist folgendes plausibel.

- Die Anzahl der Durchläufe des *loops* ist am größten für zwei *teilerfremde* a, b, d.h. wenn $\mathrm{GCD}(a, b) = 1$ gilt.
- Der Abstand $|a - b|$ verringert sich in jedem Durchlaufs des *loop* um $\min\{a, b\}$, das *Minimum* von a, b.

Somit sollt für $\mathrm{GCD}(100, 1)$ der *loop* am häufigsten durchlaufen werden, konkret 99-mal.

Als Hypothese kann man jetzt die Laufzeit von $GCD(a, b)$, für beliebige *integer* $a, b > 0$, mit $\max\{a, b\}$ (*Maximum* von a, b) als $4 \cdot \max\{a, b\} + 2$ abschätzen.

- *init* benötigt zwei *assignment operators*
- *iterate* benötigt für jeden *loop* zwei *comparison operators*, einen *arithmetic operator* und einen *assignment operator*

Euklidischer Algorithmus (modern)

Die moderne Form des Euklidischen Algorithmus kann man aus einer *trace table* für variablen *input* ableiten.

trace table mit *input* $a > b$

Zeile	x	y	Bemerkung
3, 4	a	b	
5	$a - b$	b	
⋮	⋮	⋮	
5	$r_1 = a - q_1 \cdot b$	b	r_1 = Rest bei Division $a : b$
3, 4	r_1	b	$r_1 < b$
7	r_1	$b - r_1$	
⋮	⋮	⋮	
7	r_1	$r_2 = b - q_2 \cdot r_1$	r_2 = Rest bei Division $b : r_1$
3,4	r_1	r_2	$r_2 < r_1$
5	$r_1 - r_2$	r_2	
⋮	⋮	⋮	

Euklidischer Algorithmus (modern)

input natürliche Zahlen a, b

init $x \leftarrow a, y \leftarrow b$

iterate solange $y \neq 0$

- $r \leftarrow x \bmod y$
- $x \leftarrow y$
- $y \leftarrow r$

finalize liefere x

output x

$x \bmod y$ (gesprochen: „x modulo y“): Formel für Divisionsrest von x durch y

```
1  x = a                    # input a
2  y = b                    # input b
3  while (y != 0):
4      t = x % y            # remainder of integer division
5      x = y
6      y = t
7  print(x)                 # output
```

Bemerkung

Es gilt `x % y = x` mit `x < y`. Deshalb ist eine Fallunterscheidung nicht nötig.

Sprachumfang von Python

Bislang haben wir nur mit (einem kleinen Teil) der *core language* (Kernsprache) von Python gearbeitet, z.B. mit Kontrollstrukturen und arithmetischen Operationen.

Darüber hinaus gibt es noch die *standard library* (Standardbibliothek). Das ist ein großer Fundus zusätzlicher Funktionen und Datenstrukturen, die in Modulen (z.B. `math`, `random`, `datetime`) organisiert sind. Die *standard library* ist nicht Teil der Kernsprache wird aber mit Python ausgeliefertes wird.

GCD-Problem mit *standard library*

Wenn man die Spezifikation des GCD-Problems etwas erweitert, ist ein kompakterer Algortihmus möglich.

GCD-Problem

input *integer* $a, b > 0$
output *integer* $d > 0$ mit folgenden Eigenschaften
- d teilt a und b
- es gibt keinen Teiler d' von a und b mit $d' > d$

tools Python: *core language*, *standard library*

```
1 import math
2 print (math.gcd(a, b))
```

A Anhang

A.1 Darstellung von Fachbegriffen

In diesen Vorlesungsunterlagen wird eine Mischung aus **deutscher Sprache** und **englischen Fachbegriffen** verwendet.

Englisch ist die Sprache der Informatik und der Programmierung, deshalb orientieren sich die Materialien an den in Python und in der Fachliteratur üblichen Bezeichnungen.

Grundprinzip

Beim **ersten Auftreten** eines Begriffs werden englische und deutsche Bezeichnung gemeinsam genannt. Danach wird nur noch der englische Begriff verwendet.

Schreibweise

- klein, *kursiv*: Code-nahe Begriffe (*loop*, *string*, *list*, *tuple*, *dictionary*)
- groß, normal: Abstrakte Datenstrukturen (Stack, Queue, Tree, Graph), die keine direkte Code-Entsprechung haben.

Grammatik

Begriffe werden wie deutsche Substantive behandelt (z.B. die *loops*, ein *string*).

Beispiel

Eine *loop* (Schleife) wiederholt Anweisungen, solange eine Bedingung erfüllt ist. In Python gibt es verschiedene Formen von *loops*, z.B. `for`- und `while`-*loops*.

A.2 Python Operators

Python Arithmetic Operators

Die *arithmetic operators* arbeiten mit numerischen Werten und führen grundlegende mathematische Operationen aus.

Operator	Operation	Beispiel
`+`	Addition	`x + y`
`-`	Subtraktion	`x - y`
`*`	Multiplikation	`x * y`
`/`	Division	`x / y`
`**`	Potenzieren	`x ** y`
`//`	ganzzahlige Division	`x // y`
`%`	Modulo (Rest der ganzzahligen Division)	`x % y`

Python Assignment Operators

Um Variablen Werte zuzuweisen werden *assignment operators* verwendet.

Operator	Beispiel	Entsprechung
`=`	`x = 5`	`x = 5`
`+=`	`x += 3`	`x = x + 3`
`-=`	`x -= 3`	`x = x - 3`
`*=`	`x *= 3`	`x = x * 3`
`/=`	`x /= 3`	`x = x / 3`
`**=`	`x **= 3`	`x = x ** 3`
`//=`	`x //= 3`	`x = x // 3`
`%=`	`x %= 3`	`x = x % 3`
`&=`	`x &= 3`	`x = x & 3`
`|=`	`x |= 3`	`x = x | 3`
`^=`	`x ^= 3`	`x = x ^ 3`
`>>=`	`x >>= 3`	`x = x >> 3`
`<<=`	`x <<= 3`	`x = x << 3`

Python Comparison Operators

Mit *comparison operators* kann man zwei Werte miteinander vergleichen.

Operator	Operation	Beispiel
`==`	gleich	`x == y`
`!=`	ungleich	`x != y`
`>`	größer	`x > y`
`<`	kleiner	`x < y`
`>=`	größer gleich	`x >= y`
`<=`	kleiner gleich	`x <= y`

Python Logical Operators

Um *conditions* (Bedingungen) zu verknüpfen werden *logical operators* verwendet.

Operator	Beschreibung	Beispiel
`and`	ergibt `True`, wenn beide Bedingungen wahr sind	`x < 5 and x < 10`
`or`	ergibt `True`, wenn mindestens eine Bedingung wahr ist	`x < 5 or x < 4`
`not`	kehrt das Ergebnis um, ergibt `False`, wenn die Bedingung wahr ist	`not(x < 5 and x < 10)`

Python Identity Operators

Mit *identity operators* kann man feststellen, ob zwei Referenzen auf dasselbe Objekt, am selben Speicherort, verweisen.

Operator	Beschreibung	Beispiel
`is`	ergibt `True`, wenn beide Variablen dasselbe Objekt referenzieren	`x is y`
`is not`	ergibt `True`, wenn beide Variablen unterschiedliche Objekte referenzieren	`x is not y`

Python Membership Operators

Ob ein Objekt in einer *sequence* (sequentieller Datentyp) enthalten ist, kann mit *membership Operators* überprüft werden.

Operator	Beschreibung	Beispiel
`in`	ergibt `True`, wenn eine *sequence* den angegebenen Wert enthält	`x in y`
`not in`	ergibt `True`, wenn eine *sequence* den angegebenen Wert nicht enthält	`x not in y`

Python Bitwise Operators

Zahlen können *bitwise* (bitweise) mit *bitwise operators* vergleichen werden.

Operator	Beschreibung	Beispiel
`&`	*bitwise AND* (bitweises UND): ergibt 1, wenn beide Bits 1 sind	`x & y`
`|`	*bitwise OR* (bitweises ODER): ergibt 1, wenn eines der Bits 1 ist	`x | y`
`^`	*bitwise XOR* (bitweises XOR): ergibt 1, wenn genau eines der Bits 1 ist	`x ^ y`
`~`	*bitwise NOT* (bitweise Negation): invertiert alle Bits	`~x`
`<<`	*left shift* (Linksverschiebung): verschiebt Bits nach links und füllt rechts mit Nullen auf, die Bits ganz links fallen weg	`x << 2`
`>>`	*signed right shift* (vorzeichenbehaftete Rechtsverschiebung): verschiebt Bits nach rechts und füllt links mit Kopien des ganz linken Bits auf, die Bits ganz rechts fallen weg	`x >> 2`

Python Operator Precedence

Die Reihenfolge, in der Operatoren in einem Ausdruck ausgeführt werden, wird durch *operator precedence* (Operatorpriorität) beschrieben. Die Priorität in der Tabelle nimmt von oben nach unten ab.

Operator	Beschreibung
`()`	Einklammern
`**`	Potenzieren
`*`, `/`, `//`, `%`	Multiplikation, Division, ganzzahlige Division, Modulo
`+`, `-`	Addition, Subtraktion
`+x`, `-x`, `~x`	unäre Vorzeichen, *bitwise NOT*
`<<`, `>>`	*bitwise shift left*, *bitwise shift right*
`&`	*bitwise AND*
`^`	*bitwise XOR*
`|`	*bitwise OR*
`==`, `!=`, `>`, `<`, `>=`, `<=`, `is`, `is not`, `in`, `not in`	*comparisons*, *identity*, *membership*
`not`	*logical NOT*
`and`	*logical AND*
`or`	*logical OR*

A.3 Python Keywords

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Schlagwortverzeichnis

`datetime`, 17
`def`, 9
`math`, 17
`random`, 17
`return`, 9
`sum`, 6
`while`, 6
Ablaufprotokoll, siehe trace table
Aktualisieren, siehe update
Ausgabe, siehe output
Dreieckszahl, siehe triangular number
EVA-Prinzip, siehe IPO model
Eingabe-Verarbeitung-Ausgabe Prinzip, siehe Input–Process–Output model
Eingabe, siehe input
Fortführbedingung, siehe loop condition
Funktion, siehe function
Kernsprache, siehe core language
Kopieren und Einfügen, siehe copy and paste
Rückgabewert, siehe return value
Schleifeninvariante, siehe loop invariant
Schleifen, siehe loops
Schlüsselwort, siehe keyword
Standardbibliothek, siehe standard library
Verarbeitungvorschrift, siehe process
Vergleichsoperatoren, siehe comparison operators
Werkzeuge, siehe tools
Zuweisungsoperatoren, siehe assignment operators
arithmetische Operatoren, siehe arithmetic operators
ganze Zahlen, siehe integer
größte gemeinsame Teiler, siehe greatest common divisor

Addition, 20
Algorithmus, 1, 3
Analytical Engine, 2
AND, bitwise, 22
AND, logical, 23
arithmetic operators, 7, 20
assignment operator, 7
assignment operators, 7, 20

bitwise, 22
bitwise AND, 23
bitwise NOT, 23
bitwise operators, 22
bitwise OR, 23
bitwise shift left, 23
bitwise shift right, 23
bitwise XOR, 23

comparison operators, 7, 20
comparisons, 23
conditions, 21
copy and paste, 8
core language, 17, 18

dictionary, 19
Division, 20

function, 9

ganzzahlige Division, 20
Gaußsche Summenformel, 8
GCD, 10
greatest common divisor, 10
GWDG Jupyter Cloud, 3

identity, 23
identity operators, 21
Informatik, 1
init, 16
input, 3, 7, 9, 10, 14, 16
Input–Process–Output model, 4
integer, 12, 16, 18
IPO model, 4
iterate, 14, 16

Jupyter Notebook, 3

keyword, 9
korrekt, 9
Korrektheit, 7, 9

Laufzeit, 7, 9
left shift, 22
list, 19
logical operators, 21
loop, 6, 14–16, 19
loop condition, 6
loop invariant, 14, 15
loops, 6, 10, 15

Maximum, 16
membership, 23
membership Operators, 21
Minimum, 15
Modulo, 20
Multiplikation, 20

NOT, bitwise, 22
NOT, logical, 23

operator precedence, 22
OR, bitwise, 22
OR, logical, 23
output, 3, 6, 9, 10, 15

Partiell korrekt, 15
partiell korrekt, 9
Potenzieren, 20
Problem, 3
process, 3, 7
Programm, 2
Programmcode, 12
Pseudocode, 11
Python, 3

return value, 9
right shift, signed, 22

sequence, 21, 22
Speicherbedarf, 7, 9
Spezifikation, 9
standard library, 17, 18
string, 19
Subtraktion, 20

teilerfremd, 15
terminiert, 9
Terminiert., 15
tool, 3
tools, 10
trace table, 13, 16
triangular number, 8
tuple, 19

update, 15

XOR, bitwise, 22